

Divisão de Tasks do Projeto

Sistema Escola/Curso - Programação Orientada a Objetos

Este documento organiza as responsabilidades da equipe com base no diagrama UML final e nos requisitos do projeto: código Java compilável, uso de POO, herança, polimorfismo, interfaces, coleções, exceções, try-catch, Git/GitHub e documentação.

Task	Responsável por	Entregas principais
Task 1	Main, menus e integração inicial	Main.java executável, menus, switch-case e try-catch
Task 2	Pessoa, Aluno e Professor	Herança com Pessoa, exibirDados(), matrícula e notas
Task 3	ConteudoEducacional, Curso e Disciplina	Herança acadêmica, List, Set e Gerenciavel
Task 4	Escola e Turma	List, List, Map e Gerenciavel
Task 5	Interfaces, Matrícula, Exceptions e validações	Gerenciavel, Avaliavel, exceptions customizadas e regras de erro
Task 6	Repository, Service, documentação e GitHub	Repositories, Services, README, docs e Google Sites

Fluxo geral de camadas

A estrutura recomendada separa responsabilidades para facilitar a implementação, os testes e a avaliação do código.

Usuário -> Main -> Service -> Repository -> Model

Main: menus e entrada do usuário. Service: regras de negócio. Repository: armazenamento em memória. Model: classes do diagrama UML.

Task 1 - Main, menus e integração inicial

Responsável por: criar a entrada do sistema, montar os menus e deixar o programa executável enquanto as outras partes ainda estão em desenvolvimento.

Arquivos/classes principais:

```
src/main/Main.java
src/model/
src/service/
src/repository/
src/exception/
src/util/
```

O que fazer:

- Criar a estrutura de pacotes do projeto: model, service, repository, exception, util e main.
- Criar o arquivo Main.java com o método public static void main(String[] args).
- Criar o menu principal com opções para cursos, disciplinas, turmas, alunos, professores, matrículas e notas.
- Criar submenus com switch-case para cada área do sistema.
- Deixar mensagens provisórias nas funções ainda não implementadas, como "Função cadastrar curso em desenvolvimento".

- Depois da conclusão dos Services, substituir as mensagens provisórias por chamadas reais, como `cursoService.cadastrarCurso()`.

Try-catch / validações:

- Usar `NumberFormatException` para tratar letras ou textos digitados onde o sistema espera número.
- Garantir que o sistema não feche quando o usuário digitar uma opção inválida.
- Manter o Main compilável antes mesmo das outras classes ficarem prontas.

Task 2 - Pessoa, Aluno e Professor

Responsável por: implementar as classes relacionadas às pessoas do sistema, aplicando herança e polimorfismo.

Arquivos/classes principais:

```
src/model/Pessoa.java
src/model/Aluno.java
src/model/Professor.java
```

O que fazer:

- Criar Pessoa como classe abstrata.
- Adicionar em Pessoa os atributos id, nome, email, cpf e dataNascimento.
- Criar o método `exibirDados()` em Pessoa.
- Criar Aluno herdando de Pessoa.
- Criar Professor herdando de Pessoa.
- Sobrescrever `exibirDados()` em Aluno e Professor.
- Em Aluno, implementar `matricula`, `realizarMatricula()` e `consultarMatriculas()`.
- Em Professor, implementar `especialidade`, `lançarNota()` e `listarNotas()`.
- Fazer Professor implementar a interface `Avaliavel` quando ela estiver pronta.

Try-catch / validações:

- Preparar `lançarNota()` para lançar `NotaInvalidaException` quando a nota for menor que 0 ou maior que 10.
- Não concentrar try-catch nessa task; o tratamento final será feito no Main ou nos Services.

Task 3 - ConteudoEducacional, Curso e Disciplina

Responsável por: implementar a estrutura acadêmica principal do sistema, aplicando herança, interface e coleções.

Arquivos/classes principais:

```
src/model/ConteudoEducacional.java
src/model/Curso.java
src/model/Disciplina.java
```

O que fazer:

- Criar ConteudoEducacional como classe abstrata.
- Adicionar em ConteudoEducacional os atributos id, nome e cargaHoraria.
- Criar Curso herdando de ConteudoEducacional.
- Criar Disciplina herdando de ConteudoEducacional.
- Em Curso, implementar `valorMensal`, `disciplinas`: List e ativo.

- Em Curso, implementar adicionarDisciplina(), listarDisciplinas(), atualizarDisciplina(), removerDisciplina(), ativar(), desativar(), isAtivo() e exibirDados().
- Em Disciplina, implementar descricao, turmas: List, professoresAptos: Set e ativo.
- Em Disciplina, implementar adicionarTurma(), listarTurmas(), atualizarTurma(), removerTurma(), adicionarProfessorApto(), listarProfessoresAptos(), removerProfessorApto(), ativar(), desativar(), isAtivo() e exibirDados().
- Fazer Curso e Disciplina implementarem Gerenciavel quando a interface estiver pronta.

Try-catch / validações:

- Validar dados básicos nos métodos, mas deixar exceptions principais para Services/Main quando necessário.

Task 4 - Escola e Turma

Responsável por: implementar a classe da escola e a estrutura das turmas, incluindo listas, mapa de alunos e status ativo.

Arquivos/classes principais:

```
src/model/Escola.java
src/model/Turma.java
```

O que fazer:

- Criar Escola com id, nome, endereco, telefone e cursos: List.
- Em Escola, implementar adicionarCurso(), listarCursos(), atualizarCurso() e removerCurso().
- Criar Turma com id, codigo, turno, sala, matriculas: List, alunos: Map e ativo.
- Em Turma, implementar definirProfessor(), listarAlunos(), removerAluno(), atualizarDados(), ativar(), desativar() e isAtivo().
- Fazer Turma implementar Gerenciavel quando a interface estiver pronta.

Try-catch / validações:

- Preparar os métodos para funcionarem com validações dos Services.
- Não colocar toda a regra no model; regras como turma inexistente ou aluno duplicado podem ser tratadas no Service.

Task 5 - Interfaces, Matrícula, Exceptions e validações

Responsável por: implementar os conceitos avançados mais importantes para a avaliação: interfaces, exceções customizadas e regras de validação.

Arquivos/classes principais:

```
src/model/Gerenciavel.java
src/model/Avaliavel.java
src/model/Matricula.java
src/exception/NotaInvalidaException.java
src/exception/MatriculaInvalidaException.java
```

O que fazer:

- Criar a interface Gerenciavel com ativar(), desativar() e isAtivo().
- Criar a interface Avaliavel com lancarNota() e listarNotas().
- Garantir que Curso, Disciplina e Turma implementem Gerenciavel.
- Garantir que Professor implemente Avaliavel.
- Criar Matricula com id, dataMatricula, status e notaFinal.

- Em Matricula, implementar confirmar() e cancelar().
- Criar NotaInvalidaException.
- Criar MatriculaInvalidaException.

Try-catch / validações:

- Lançar NotaInvalidaException para nota menor que 0 ou maior que 10.
- Lançar MatriculaInvalidaException para aluno nulo, turma nula, matrícula duplicada ou matrícula cancelada sendo confirmada novamente.
- Não é obrigatório criar AlunoNaoEncontradoException; esse caso pode ser tratado como MatriculaInvalidaException quando ocorre no processo de matrícula.

Task 6 - Repository, Service, documentação e GitHub

Responsável por: implementar armazenamento em memória, regras de negócio, integração com o Main e documentação do projeto.

Arquivos/classes principais:

```
src/repository/CursoRepository.java
src/repository/DisciplinaRepository.java
src/repository/TurmaRepository.java
src/repository/AlunoRepository.java
src/repository/ProfessorRepository.java
src/repository/MatriculaRepository.java
src/service/CursoService.java
src/service/DisciplinaService.java
src/service/TurmaService.java
src/service/AlunoService.java
src/service/ProfessorService.java
src/service/MatriculaService.java
src/service/NotaService.java
```

O que fazer:

- Criar repositories para salvar, listar, buscarPorId, atualizar e remover objetos em memória.
- Usar listas internas nos repositories, como List, List e List.
- Criar services para aplicar regras de negócio antes de salvar ou alterar dados.
- Services devem buscar objetos nos repositories, validar dados e chamar métodos dos models.
- Preparar os services para serem chamados pelo Main.
- Atualizar README.md com instruções de execução.
- Organizar docs/diagrama-uml/v1, v2, v3 etc.
- Documentar funcionalidades, decisões técnicas e prints do sistema funcionando.
- Ajudar a manter o Google Sites atualizado com progresso, documentos e resultados.

Try-catch / validações:

- Services podem lançar exceptions para o Main tratar com try-catch.
- Exemplo: MatriculaService lança MatriculaInvalidaException e NotaService lança NotaInvalidaException.

Ordem recomendada de execução

- Criar primeiro os esqueletos das classes e interfaces para evitar que uma pessoa trave a outra.
- Task 1 deixa o Main executável com menus provisórios.
- Tasks 2, 3, 4 e 5 implementam os Models, interfaces e exceptions.
- Task 6 cria repositories e services usando os models prontos.
- Task 1 integra o Main com os Services.
- Todos testam o fluxo completo: cadastrar curso, disciplina, turma, aluno, professor, matrícula e nota.
- Finalizar documentação, README, histórico do diagrama, Google Sites e apresentação.

Checklist final por requisito

Requisito	Onde aparece no projeto
Herança	Aluno e Professor extends Pessoa; Curso e Disciplina extends ConteudoEducacional
Polimorfismo	exibirDados() sobrescrito nas classes filhas
Classes abstratas	Pessoa e ConteudoEducacional
Interfaces	Gerenciavel e Avaliavel
Coleções	List, List, List, List, Set, Map
Exceções customizadas	NotaInvalidaException e MatriculaInvalidaException
Try-catch	Main tratando entrada inválida e operações de matrícula/nota
Código organizado	Pacotes model, service, repository, exception, util e main
Git/GitHub	Commits semânticos e versionamento do projeto
Documentação	README, docs/diagrama-uml e Google Sites

Observação importante

As tasks não são totalmente independentes, porque as classes se relacionam. Para evitar atrasos, cada pessoa deve criar primeiro os arquivos/esqueletos das classes com os nomes e métodos combinados. Assim, o restante do grupo consegue importar e continuar trabalhando sem quebrar o projeto.